

Student CRUD Project

Step-by-Step Coding Notes

Notes based on the supplied Student CRUD code. The organization, compact spacing, code-block treatment, tables, terminology sections, quick revision, and interview section follow the style of the supplied Spring Core Fundamentals PDF.

Table of Contents

1. Project Structure
2. Student Entity
3. Controller Layer
4. Create Student — POST
5. Service Layer
6. Repository Layer
7. Read Student by ID — GET
8. Read All Students — GET
9. Update Student — PUT
10. Soft Delete — PATCH
11. Permanent Delete — DELETE
12. Spring Boot Application
13. application.properties
14. Complete CRUD Flow
15. Important Terminology
16. Code Observations
17. Quick Revision
18. Interview Questions
19. Final Mental Model

1. Project Structure

The project is divided into layers so that each part has a clear responsibility.

```
com.example.student
├── entity
│   └── Student
├── StudentController
│   └── Controller
├── StudentRepository
│   └── StudentRepository
├── StudentService
│   └── Service
├── StudentApplication
├── resources
│   └── application.properties
```

Layer responsibilities

Layer	Responsibility
Entity	Represents the Student data and database mapping.
Controller	Receives HTTP requests and returns HTTP responses.
Service	Contains application/business logic.
Repository	Performs persistence/database operations.
Application	Starts the Spring Boot application.
Properties	Contains server, MySQL and JPA configuration.

The main request flow is:

```
Client / Postman
    ↓
Controller
    ↓
Service
    ↓
Repository
    ↓
MySQL Database
```

2. Student Entity

The Student class is the model used by the application and is also mapped to a database table through JPA.

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String studentName;
    private int age;
    private int rollNo;
    private boolean isDeleted;
    private String email;
    private String subject;
}
```

@Entity

@Entity tells JPA that Student is a persistent entity. JPA/Hibernate can map its fields to columns in the database.

@Id

@Id marks id as the primary-key field. The primary key identifies a Student record.

@GeneratedValue

GenerationType.IDENTITY tells JPA to use the database's identity/auto-increment mechanism for generating the ID.

Student Fields

Field	Purpose
id	Primary key.
studentName	Student's name.
age	Student's age.
rollNo	Student's roll number.
isDeleted	Stores soft-delete state.
email	Student's email.
subject	Student's subject.

Getters and Setters

Getters read values from fields. Setters change field values. For example, `setDeleted(false)` marks a newly created record as active. Code note: the supplied Student class imports `UuidGenerator`, but the annotation is not used. The unused import can be removed.

3. Controller Layer

The controller is the HTTP entry point. It maps URLs and HTTP methods to Java methods.

```
@RestController
@RequestMapping("/api/student")
public class Controller {

    private Service service;

    public Controller(Service service) {
        this.service = service;
    }
}
```

@RestController

Marks the class as a REST controller. Its methods handle HTTP requests and return response data.

@RequestMapping

`@RequestMapping("/api/student")` defines the common base path for this controller.

```
GET /api/student/{id}
GET /api/student/all
POST /api/student
PUT /api/student/update/{id}
PATCH /api/student/soft-delete/{id}
DELETE /api/student/delete/{id}
```

Constructor Injection

The Controller requires Service. Spring provides the Service object through the constructor.

```
public Controller(Service service) {
    this.service = service;
}
```

Dependency relationship:

```
Controller
↓
Service
```

The controller should mainly handle HTTP concerns. The actual application logic is delegated to the service.

4. Create Student — POST

```
@PostMapping("")
public ResponseEntity<Student> createStudent(
    @RequestBody Student student) {

    Student studentResult = service.addStudent(student);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(studentResult);
}
```

Step 1 — @PostMapping

Maps a POST request to `/api/student`.

Step 2 — @RequestBody

`@RequestBody` takes JSON from the request body and converts it into a Student object.

```
POST /api/student

{
  "studentName": "Rahul",
  "age": 21,
  "rollNo": 10,
  "email": "rahul@example.com",
  "subject": "Java"
}
```

Step 3 — Service Call

service.addStudent(student) sends the Student object to the service layer.

Step 4 — ResponseEntity

ResponseEntity allows the controller to specify the HTTP status and response body.

The supplied code returns 201 CREATED for a successful create operation.

5. Service Layer

The service layer contains the application logic between the controller and repository.

```
@org.springframework.stereotype.Service
public class Service {

    private final StudentRepository studentRepository;

    public Service(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }
}
```

@Service

@Service registers the class as a Spring-managed service bean.

Repository Dependency

The Service depends on StudentRepository. Constructor injection supplies this dependency.

```
Service
  ↓
StudentRepository
```

Create Logic

```
public Student addStudent(Student student) {
    student.setDeleted(false);

    Student result = studentRepository.save(student);

    System.out.println("reached till service");
    return result;
}
```

First, isDeleted is set to false. Then repository.save() persists the Student. The saved object is returned to the controller.

6. Repository Layer

The repository provides the database-access abstraction.

```
@Repository
public interface StudentRepository
    extends JpaRepository<Student, Long> {

    Optional<Student> findByIdAndIsDeletedIsFalse(Long id);

    Student[] findByAndIsDeletedFalse();
}
```

JpaRepository<Student, Long>

Student is the entity type and Long is the type of its primary key.

JpaRepository already provides common operations such as save(), findById(), existsById() and deleteById().

Derived Query Method

```
findByIdAndIsDeletedIsFalse(Long id)
```

Spring Data JPA interprets the method name and creates a query that matches the ID and requires isDeleted to be false.

Optional<Student>

Optional represents that a Student may or may not be found.

```
Optional<Student> result = repository.findByIdAndIsDeletedIsFalse(id);

if (result.isPresent()) {
    Student student = result.get();
}
```

Second Repository Method

```
Student[] findByAndIsDeletedFalse();
```

The supplied code intends this method to return non-deleted students. The method name is unusual; a conventional derived-query form would normally be `findByIsDeletedFalse()`. Verify the method by compiling the project.

7. Read Student by ID — GET

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudentById(
    @PathVariable("id") Long id) {

    Student s = service.findStudentById(id);

    if (s == null || s.getDeleted().equals(true)) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .build();
    }

    return ResponseEntity
        .status(HttpStatus.OK)
        .body(s);
}
```

@GetMapping

Maps GET `/api/student/{id}` to this method.

@PathVariable

`@PathVariable` reads the value from the URL and places it into the `id` parameter.

```
GET /api/student/5
  ↓
id = 5
```

Not Found

If the service returns null, the controller returns 404 NOT_FOUND.

Success

If the Student exists, the controller returns 200 OK with the Student as the response body.

8. Read All Students — GET

```
@GetMapping("/all")
public List<Student> getAllStudent() {
    List<Student> l;
    l = service.getAllStudent();
    return l;
}
```

Endpoint: GET `/api/student/all`

Service Logic

```
public List<Student> getAllStudent() {

    List<Student> res = new ArrayList<>();

    for (Student t : studentRepository.findByAndIsDeletedFalse()) {
        if (t.getDeleted().equals(false)) {
            res.add(t);
        }
    }

    return res;
}
```

The service gets records from the repository, checks the deleted flag, adds active students to the result list, and returns the list.

Code note: if the repository query already guarantees `isDeleted=false`, the second deleted check is redundant.

9. Update Student — PUT

```
@PutMapping("/update/{id}")
public ResponseEntity<String> updateStudentDetailsById(
    @PathVariable Long id,
    @RequestBody Student s) {

    Student b = service.updateData(id, s);

    if (b == null) {
        return ResponseEntity.ok("Not Found-> " + id);
    }

    return ResponseEntity.ok("Record Updated---> " + id);
}
```

Request

PUT /api/student/update/{id} receives the target record ID from the URL and new values from the request body.

Service Update Logic

```
public Student updateData(Long id, Student s) {

    Optional<Student> existingStudent =
        studentRepository.findByIdAndIsDeletedIsFalse(id);

    if (existingStudent.isEmpty()) {
        return null;
    }

    Student updated_Student = existingStudent.get();

    updated_Student.setId(s.getId());
    updated_Student.setStudentName(s.getStudentName());
    updated_Student.setAge(s.getAge());
    updated_Student.setEmail(s.getEmail());
    updated_Student.setRollNo(s.getRollNo());
    updated_Student.setSubject(s.getSubject());

    return studentRepository.save(updated_Student);
}
```

Update sequence: find existing record → check existence → get entity → copy fields → save.

Code note: the supplied code sets the existing entity's ID using s.getId(). When updating by a path ID, the existing ID would normally be preserved. If the body contains a different ID, this can cause unintended behavior.

10. Soft Delete — PATCH

```
@PatchMapping("/soft-delete/{id}")
public ResponseEntity<String> deleteStudentById(
    @PathVariable Long id) {

    boolean b = service.deletebyId(id);

    if (b) {
        return ResponseEntity.ok("Soft-Deleted ---> " + id);
    }

    return ResponseEntity
        .accepted()
        .body("Not Found" + id);
}
```

What is Soft Delete?

Soft delete does not remove the database row. Instead, the application changes isDeleted from false to true.

Service Logic

```
public boolean deletebyId(Long id) {

    Optional<Student> j =
        studentRepository.findByIdAndIsDeletedIsFalse(id);

    if (j.isPresent()) {
        Student s = j.get();
        s.setDeleted(true);
        updateData(id, s);
        return true;
    }

    return false;
}
```

Flow: find active Student → set isDeleted=true → save changes → return true.

```

Database row
↓
isDeleted = false
↓
PATCH soft-delete
↓
isDeleted = true
↓
Row remains in database

```

11. Permanent Delete — DELETE

```

@DeleteMapping("delete/{id}")
public ResponseEntity<String> permanentDeleteStudentById(
    @PathVariable Long id) {

    boolean b = service.permanentdeletebyId(id);

    if (b) {
        return ResponseEntity.ok("Deleted---> " + id);
    }

    return ResponseEntity.ok("Not Found ! " + id);
}

```

Service Logic

```

public boolean permanentdeletebyId(Long id) {

    boolean s = studentRepository.existsById(id);

    if (s) {
        studentRepository.deleteById(id);
        return true;
    }

    return false;
}

```

existsById() checks whether the ID exists. deleteById() permanently removes the row.

Soft Delete	Permanent Delete
PATCH	DELETE
Sets isDeleted=true	Removes the row
Row remains in DB	Row is removed from DB
Can be excluded from normal reads	No row remains to read

12. Spring Boot Application

```

@SpringBootApplication
public class StudentApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentApplication.class, args);
    }
}

```

@SpringBootApplication

Marks StudentApplication as the main Spring Boot application/configuration class.

SpringApplication.run()

Starts the Spring Boot application, creates the application context, configures the application, and starts the embedded web server.

```

main()
↓
SpringApplication.run()
↓
Spring Boot starts
↓
Spring beans are created
↓
Web application becomes ready

```

13. application.properties

```
spring.application.name=student
server.port=9090

spring.datasource.url=jdbc:mysql://localhost:3306/student_crud_db
spring.datasource.username=root
spring.datasource.password=1234

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Property	Meaning
spring.application.name	Application name.
server.port	Embedded server port; here 9090.
spring.datasource.url	MySQL JDBC connection URL.
spring.datasource.username	Database username.
spring.datasource.password	Database password.
spring.jpa.hibernate.ddl-auto	Hibernate schema handling; update is used here.
spring.jpa.show-sql	Print generated SQL.
hibernate.format_sql	Format generated SQL for readability.

Security note: the supplied password is hard-coded. In a real project, database credentials should normally be externalized and not committed to source control.

14. Complete CRUD Flow

The complete application flow can be visualized as follows.

```

CREATE
POST /api/student
↓
Controller
↓
Service.addStudent()
↓
student.setDeleted(false)
↓
Repository.save()
↓
MySQL
↓
201 CREATED

READ ONE
GET /api/student/{id}
↓
Controller
↓
Service.findStudentById()
↓
Repository.findByIdAndIsDeletedIsFalse()
↓
200 OK / 404 NOT_FOUND

READ ALL
GET /api/student/all
↓
Controller → Service → Repository
↓
List<Student>
↓
JSON

UPDATE
PUT /api/student/update/{id}
↓
Controller → Service.updateData()
↓
Find → Modify → Save
↓
200 OK

SOFT DELETE
PATCH /api/student/soft-delete/{id}
↓
Find active record
↓
isDeleted = true
↓
Save
↓
200 OK

PERMANENT DELETE
DELETE /api/student/delete/{id}
↓
existsById()
↓
deleteById()
↓
Row removed

```

15. Important Terminology

Term	Meaning
Entity	Java class mapped to persistent database data.
Controller	Layer that handles HTTP requests and responses.
Service	Layer containing application/business logic.
Repository	Layer used for persistence/database operations.
JPA	Java persistence API used to map Java objects to relational data.
Hibernate	JPA implementation used by Spring Boot in this project.
JpaRepository	Spring Data interface providing repository CRUD operations.
Dependency Injection	Providing a required object from outside instead of creating it inside.
@RequestBody	Converts request JSON into a Java object.
@PathVariable	Reads a value from a URL path.
Optional	Represents a value that may or may not be present.
Soft Delete	Marks a record deleted without removing its database row.
Permanent Delete	Removes the database row.

16. Code Observations

These points are visible from the supplied code itself and are useful when debugging or improving the project.

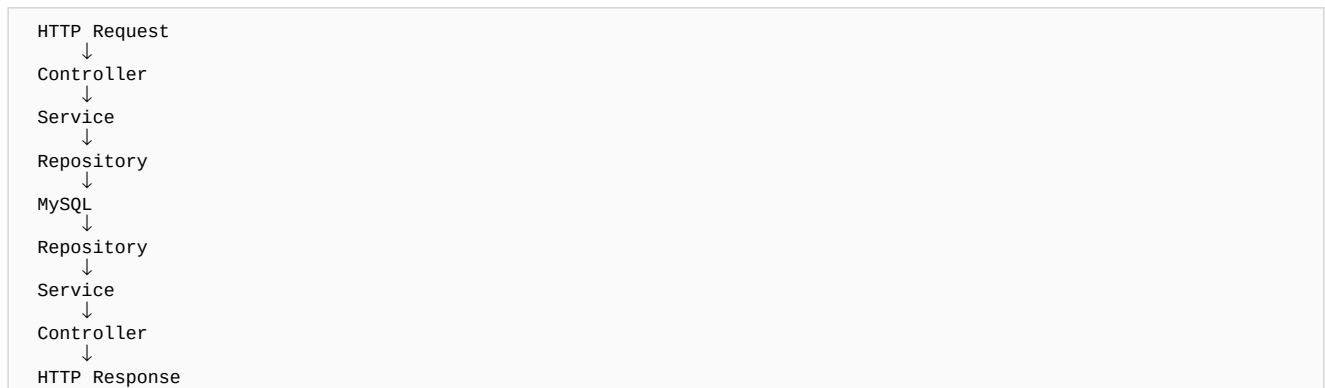
- Student imports UuidGenerator but does not use it.
- The variable boolean f=false in the update controller is unused.
- The repository method findByAndIsDeletedFalse() should be verified because its naming is unusual for a Spring Data derived query.
- The getAllStudent() service performs an additional isDeleted=false check after the repository query intends to filter deleted records.
- The update method sets updated_Student.setId(s.getId()). The path-variable id is the record being updated, so preserving the existing entity ID is normally safer.
- The update endpoint returns HTTP 200 even when its message says Not Found.
- The soft-delete method calls updateData(), which updates multiple fields. A direct save after changing only isDeleted would be simpler.
- The Student field is primitive boolean while the getter/setter use Boolean. Java performs boxing/unboxing, but using one type consistently is clearer.

These are observations about the supplied implementation, not changes made silently to the original code.

17. Quick Revision

Topic	Remember
Student	JPA entity representing student data.
@Entity	Makes the class a JPA entity.
@Id	Primary key.
@GeneratedValue	Generates ID.
@RestController	REST API controller.
@RequestMapping	Base endpoint path.
@RequestBody	JSON → Java object.
@PathVariable	URL value → Java parameter.
@Service	Business/application logic.
@Repository	Database access.
JpaRepository	Built-in CRUD repository methods.
POST	Create.
GET	Read.
PUT	Update.
PATCH	Partial/state change; here soft delete.
DELETE	Permanent deletion.
isDeleted	Soft-delete flag.
ResponseEntity	Controls response status/body.

One-line revision:



18. Interview Questions

Q1. What is the role of the Controller?

It receives HTTP requests, maps them to methods, calls the service layer, and builds the HTTP response.

Q2. Why is a Service layer used?

It keeps application/business logic separate from HTTP handling and database-access code.

Q3. What does JpaRepository provide?

It provides common persistence operations and supports derived query methods.

Q4. What does @RequestBody do?

It converts JSON request data into the Java object used by the controller method.

Q5. What does @PathVariable do?

It extracts a value from the URL path.

Q6. What is soft delete?

The row remains in the database, but a flag such as isDeleted is changed to true.

Q7. Difference between soft delete and permanent delete?

Soft delete changes the state of a row; permanent delete removes the row.

Q8. Why is Optional used in findByIdAndIsDeletedIsFalse()?

The query may return a Student or no Student, so Optional explicitly represents that possibility.

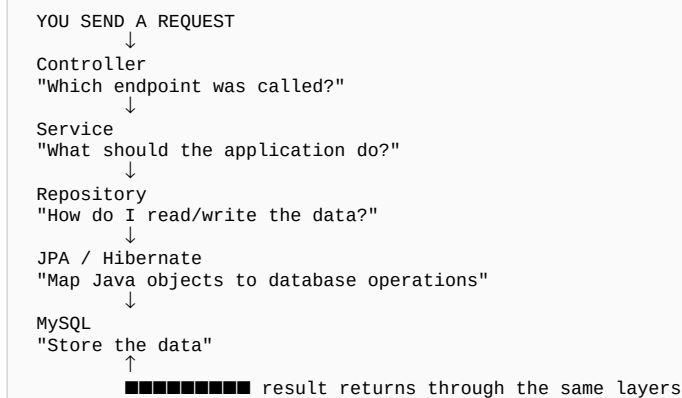
Q9. What does @GeneratedValue(strategy = GenerationType.IDENTITY) mean?

The database identity/auto-increment mechanism is used to generate the primary-key value.

Q10. Explain one POST request end-to-end.

JSON reaches the controller → @RequestBody creates Student → controller calls Service → service sets isDeleted=false → repository.save() persists it → saved Student returns through the layers → controller sends the HTTP response.

19. Final Mental Model



The most important idea to remember:

Controller handles HTTP. Service handles application logic. Repository handles persistence. Entity represents the data. Spring Boot creates and wires the application components.

For this Student CRUD project, the six operations are:

POST	→ Create Student
GET	→ Read Student(s)
PUT	→ Update Student
PATCH	→ Soft Delete
DELETE	→ Permanent Delete

End of notes.